



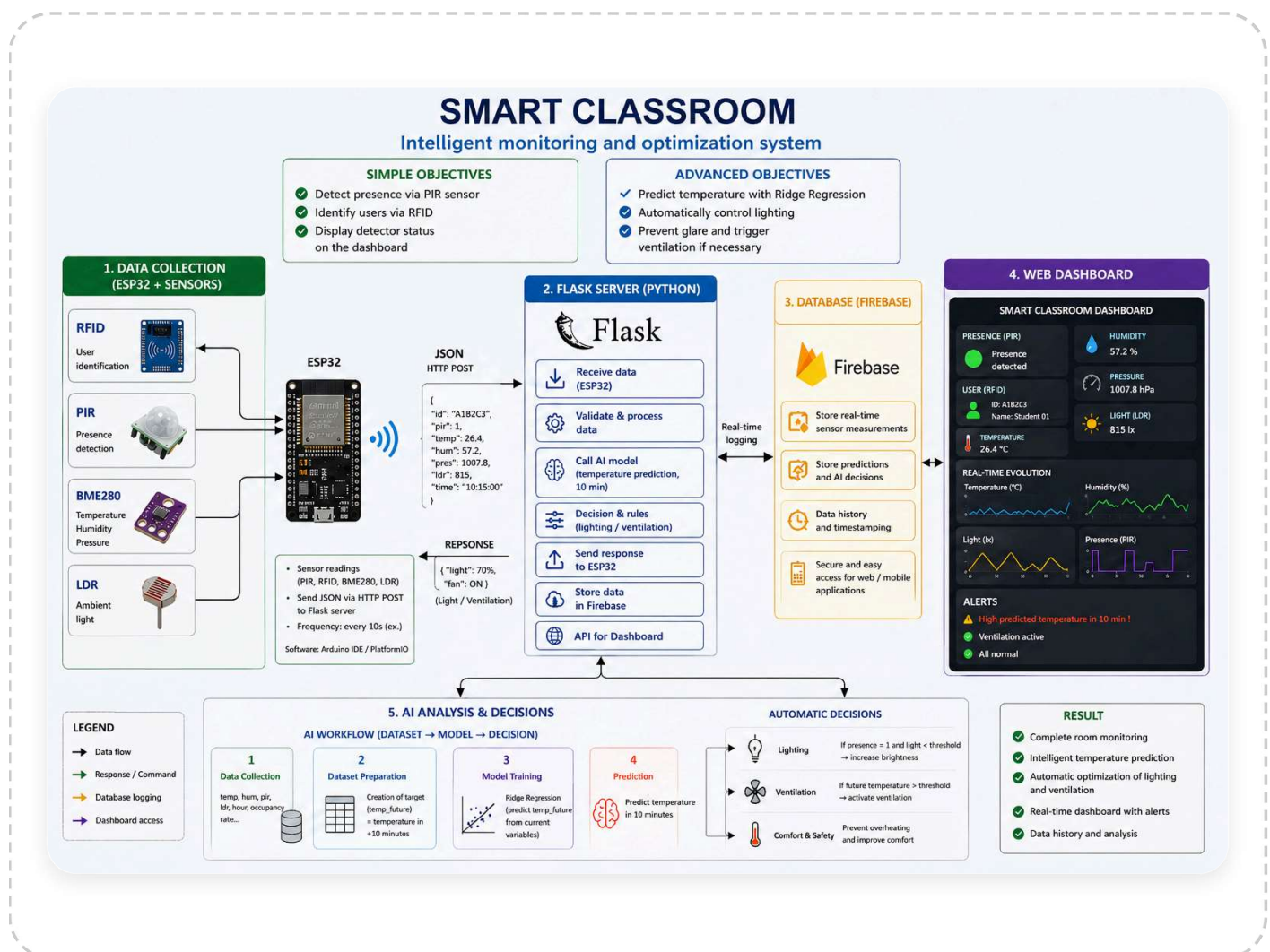
Smart Classroom – Core IoT System

ESP32 · BME280 · PIR · RFID · Flask · Firebase · ML temperature prediction · AC automation

1. Introduction

The **Smart Classroom** project provides an intelligent environment monitoring and control system for educational spaces. It collects real-time sensor data (temperature, humidity, motion, occupancy), stores it in Firebase, uses a machine learning model to predict future temperature, and automatically controls the air conditioning to improve comfort and energy efficiency. A web dashboard displays live readings, historical statistics, and allows manual AC override.

2. System Architecture



The system follows a layered architecture:

- **Hardware layer** – ESP32 reads BME280 (temperature/humidity), PIR (motion), and RFID-RC522 (people count). A relay controls the AC unit.
- **Backend layer** – Flask (Python) receives sensor data, runs the ML model, decides AC state, and serves the web frontend.
- **Database layer** – Firebase Realtime Database stores sensor history, latest readings, AC state, and configuration.
- **Frontend layer** – HTML/CSS/JS dashboard polls the backend and displays live charts and controls.
- **ML pipeline** – Offline training with SGDRegressor; the model is loaded by Flask at startup.

3. Hardware Setup

Components

- ESP32-WROOM-32 development board
- BME280 (I²C) – temperature & humidity
- HC-SR501 PIR motion sensor
- RFID-RC522 module (SPI) – people counting
- 1-channel relay module (5V) – AC control
- Jumper wires, breadboard, 5V power supply

Pin connections (Arduino firmware)

```
BME280 SDA → GPIO 25
BME280 SCL → GPIO 26
RFID SS    → GPIO 14
RFID MOSI  → GPIO 15
RFID MISO  → GPIO 12
RFID SCK   → GPIO 13
PIR OUT    → GPIO 16
Relay IN   → GPIO 17
```

Refer to the Arduino code (`esp32_firmware.ino`) for the complete implementation.

4. Software Architecture

Backend (Flask)

Key endpoints provided by `app.py` :

- `POST /data` – receives sensor readings, triggers prediction and AC decision
- `GET /ac-command` – polled by ESP32 to obtain the current AC status
- `GET /api/latest` – returns the latest sensor values and AC state for the dashboard
- `POST /api/set-ac` – admin endpoint to switch AUTO/MANUAL mode or force ON/OFF
- `POST /api/set-threshold` – changes the temperature threshold (AUTO mode)
- `GET /api/export-csv` – exports sensor history as CSV (admin only)

The Flask server maintains an in-memory `AppState` object that holds the latest readings, predicted temperature, AC status, mode, and threshold. A background thread synchronises data to Firebase every 10 seconds.

Frontend Dashboard

The dashboard (`/dashboard`) is built with HTML5, CSS3, JavaScript, and Chart.js. It polls `/api/latest` every 5 seconds, updates live tiles (temperature, humidity, motion, people count) and the temperature trend chart. An “AI Forecast” box shows the next-10-minute predicted temperature. AC control widgets are available for administrators.

Firebase Realtime Database structure

```
{
  "users": { ... },
  "sensors": {
    "latest": { temp, hum, pir, taux_occupation, predicted_temp, timestamp },
    "history": { pushId: { ... same fields } }
  },
  "devices": { "ac": { status, mode } },
  "config": { "ac_threshold": 18.0 }
}
```

5. Machine Learning Pipeline

A **SGDRegressor** (Stochastic Gradient Descent with squared error and L2 penalty) is used to predict the temperature 10 minutes ahead.

- **Features:** current temperature, humidity, PIR state (0/1), occupancy count.
- **Target:** temperature measured 10 minutes later (`temp_future`).
- **Training script** (`train_model.py`) loads three CSV datasets (`data_batch1.csv` , ..., `data_batch3.csv`), scales features with `StandardScaler` , and trains incrementally using `partial_fit` .
- **Output:** `scaler.pkl` and `temperature_model.pkl` are saved and loaded by Flask at startup.

When a new sensor reading arrives, Flask scales the features, calls `model.predict()` , and stores the result. In AUTO mode, the AC turns ON if the predicted temperature $\geq$ threshold; otherwise it turns OFF. In MANUAL mode, the admin-set status overrides the ML decision.

6. Deployment Instructions

1. Backend setup

- Install Python 3.9+ and required packages: `pip install flask firebase-admin scikit-learn pandas numpy joblib flask-cors`
- Place your `service-account-key.json` (Firebase Admin SDK) in the project root.
- Place the trained `scaler.pkl` and `temperature_model.pkl` in the root.
- Run `python app.py` – the server will start on port 5000.

2. Hardware (ESP32) setup

- Install Arduino libraries: WiFi, HTTPClient, Wire, SPI, MFRC522, Adafruit BME280, ArduinoJson.
- Open `esp32_firmware.ino` , update WiFi credentials and the Flask server IP address.
- Upload the code to ESP32.

3. Testing without hardware

Run the provided `simulate_sensors.py` script on the same machine as Flask. It generates realistic sensor data and POSTs to `/data` every 5 seconds.

7. Conclusion

The Smart Classroom system successfully integrates IoT sensing, machine learning, and real-time web technologies to create an adaptive environment. Future enhancements may include adding

CO₂ sensors, email alerts, and a mobile application. The current core system provides a solid foundation for energy-efficient and comfortable learning spaces.
